SCM Lab Report

Source Code Management

Name: Nandan Kumar C
B.Tech – Computer Science Engineering
SEN : A86605225131
Amity University, Bangalore

Introduction

1.1 What is Git?

Git is a free and open-source distributed version control system created by Linus Torvalds in 2005. It allows developers to track changes in their code, collaborate with others, and revert to earlier versions when needed. Git runs locally on your machine and does not require an internet connection.

1.2 What is Version Control?

Version control is a system that records changes to files over time so you can recall specific versions later. It allows multiple developers to work on the same project without overwriting each other's work. It also provides a complete history of every change made, making it easy to fix mistakes and understand how a project evolved.

1.3 What is GitHub?

GitHub is a cloud-based platform for hosting Git repositories online. It was founded in 2008 and is now owned by Microsoft. GitHub allows developers to store their code remotely, share it with others, collaborate through pull requests, and manage projects using built-in tools like issues and wikis.

1.4 Difference Between Git and GitHub

Although Git and GitHub are often mentioned together, they are different things:

- Git is a tool installed on your local computer, while GitHub is an online platform you access through a browser.
- Git manages version control tracking commits, branches, and merges entirely offline. GitHub stores your repository in the cloud so others can access it.
- Git was built by Linus Torvalds and is open-source. GitHub is a commercial product owned by Microsoft.
- Git is used via terminal commands. GitHub provides a visual web interface and additional collaboration features like pull requests, issue tracking, and project boards.
- You can use Git without GitHub, but GitHub always depends on Git as its underlying version control engine.

Section 2: Basic Git & Git Bash Commands

The following section documents all commands performed during the lab sessions. Each entry shows the command, a description of what it does, and a screenshot of the actual terminal output.

2.1 Git Configuration Commands

These commands configure Git with your identity. Git uses this information to tag every commit with the author's name and email.

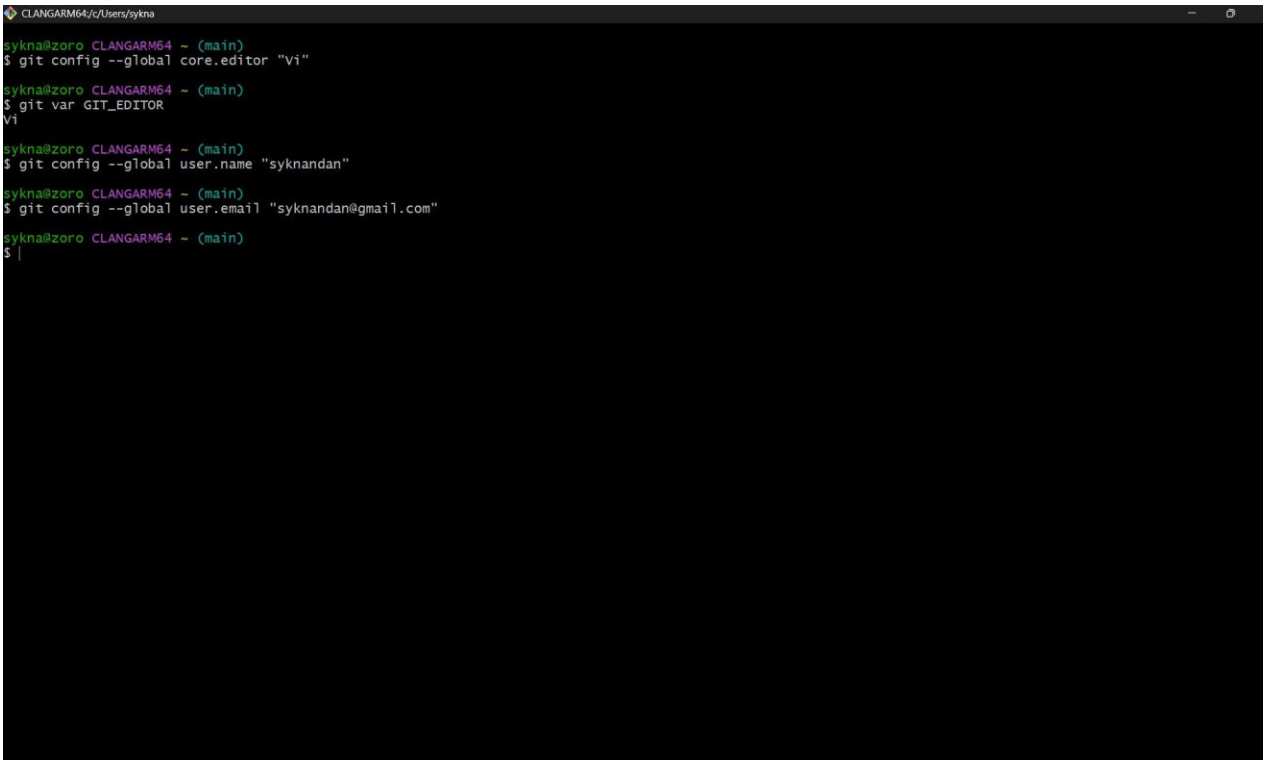
Command:

```
git config --global user.name "syknandan"
```

Description:

Sets the global username for all Git commits on this machine. The --global flag applies it to every repository.

Output (Screenshot):



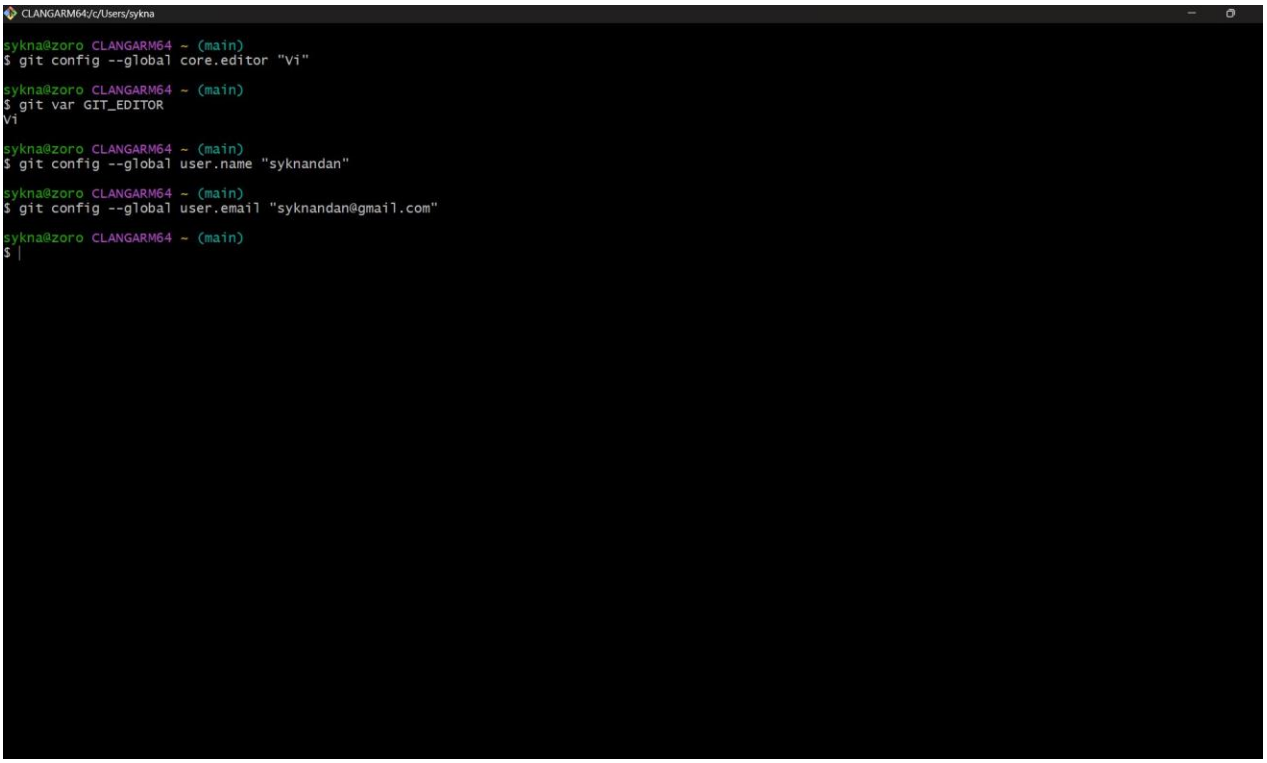
Command:

```
git config --global user.email "syknandan@gmail.com"
```

Description:

Sets the global email address associated with Git commits. This links commits to your account on platforms like GitHub.

Output (Screenshot):



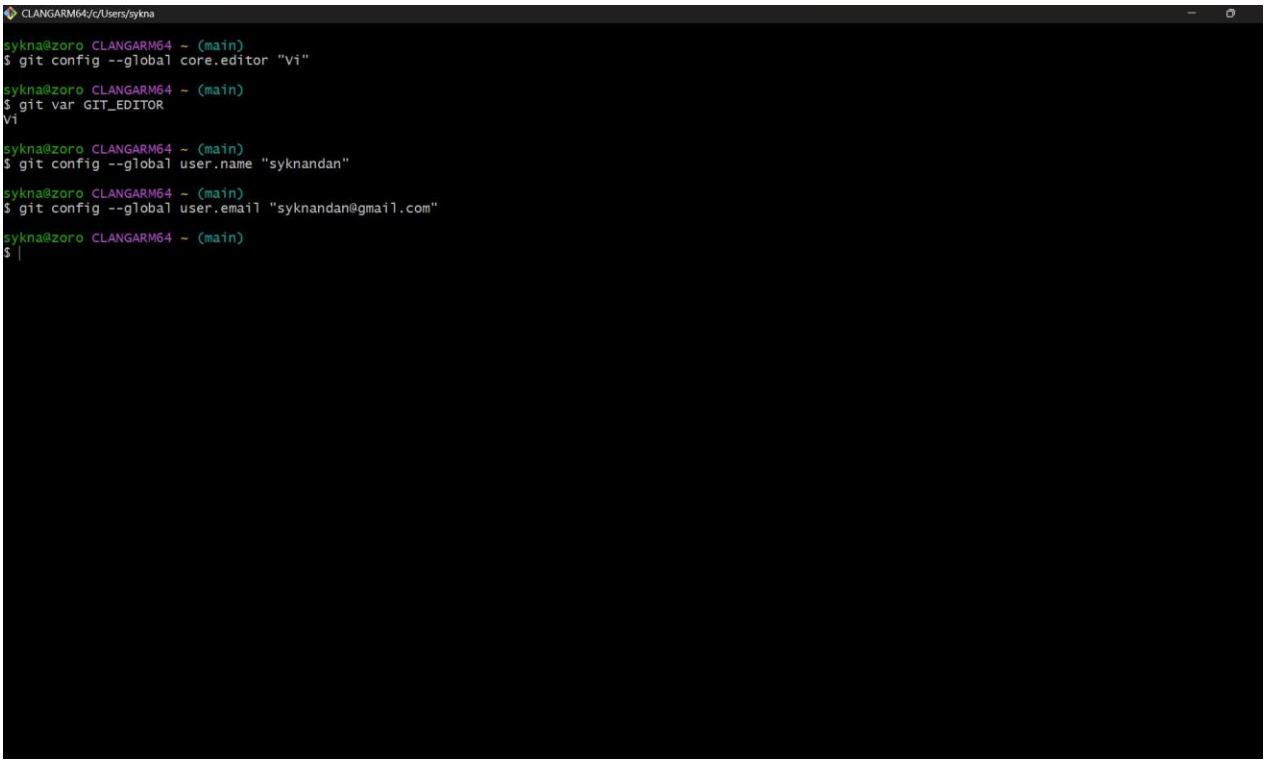
Command:

```
git config --global core.editor "vi"
```

Description:

Sets vi as the default text editor Git will use when it needs a commit message (e.g., during a merge).

Output (Screenshot):



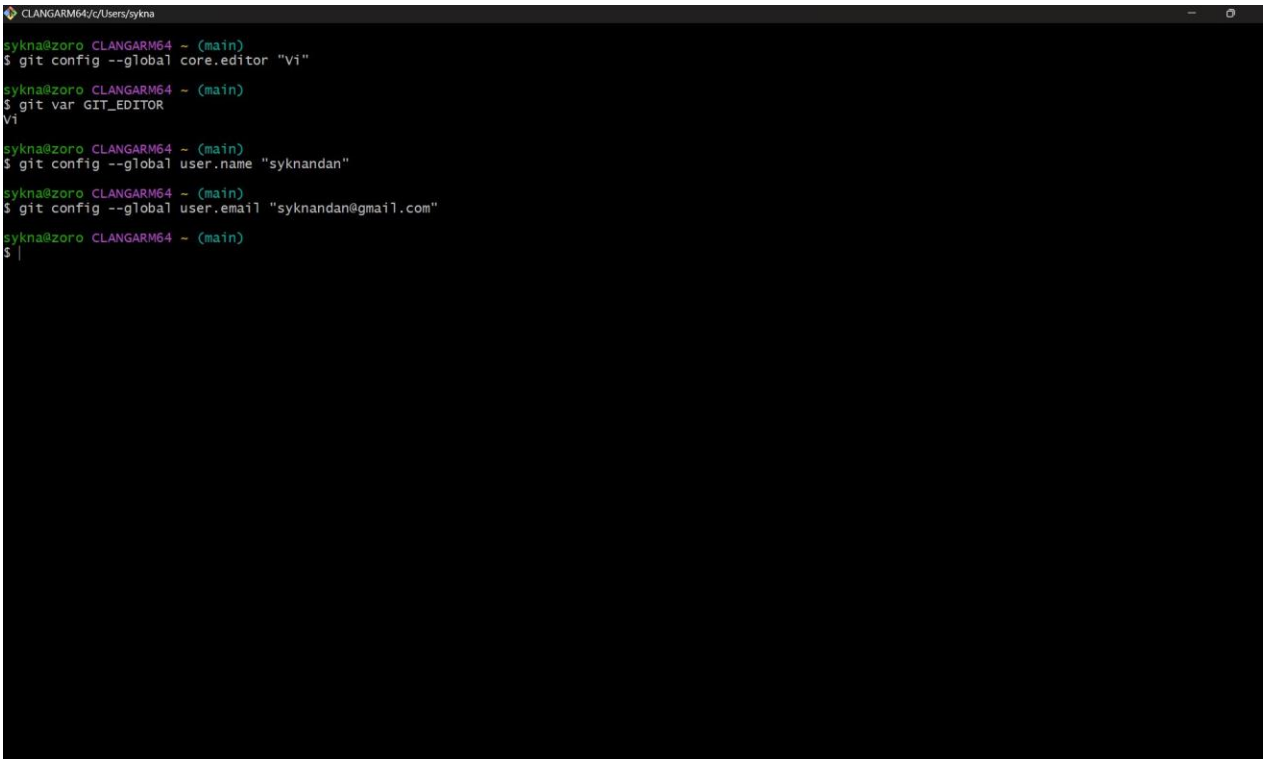
Command:

```
git var GIT_EDITOR
```

Description:

Displays the currently configured Git editor. Used here to verify that vi was set correctly.

Output (Screenshot):



2.2 Navigation Commands

Command:

```
cd newfolder1
```

Description:

Changes the current working directory to "newfolder1". All subsequent commands will run relative to this directory.

Output (Screenshot):

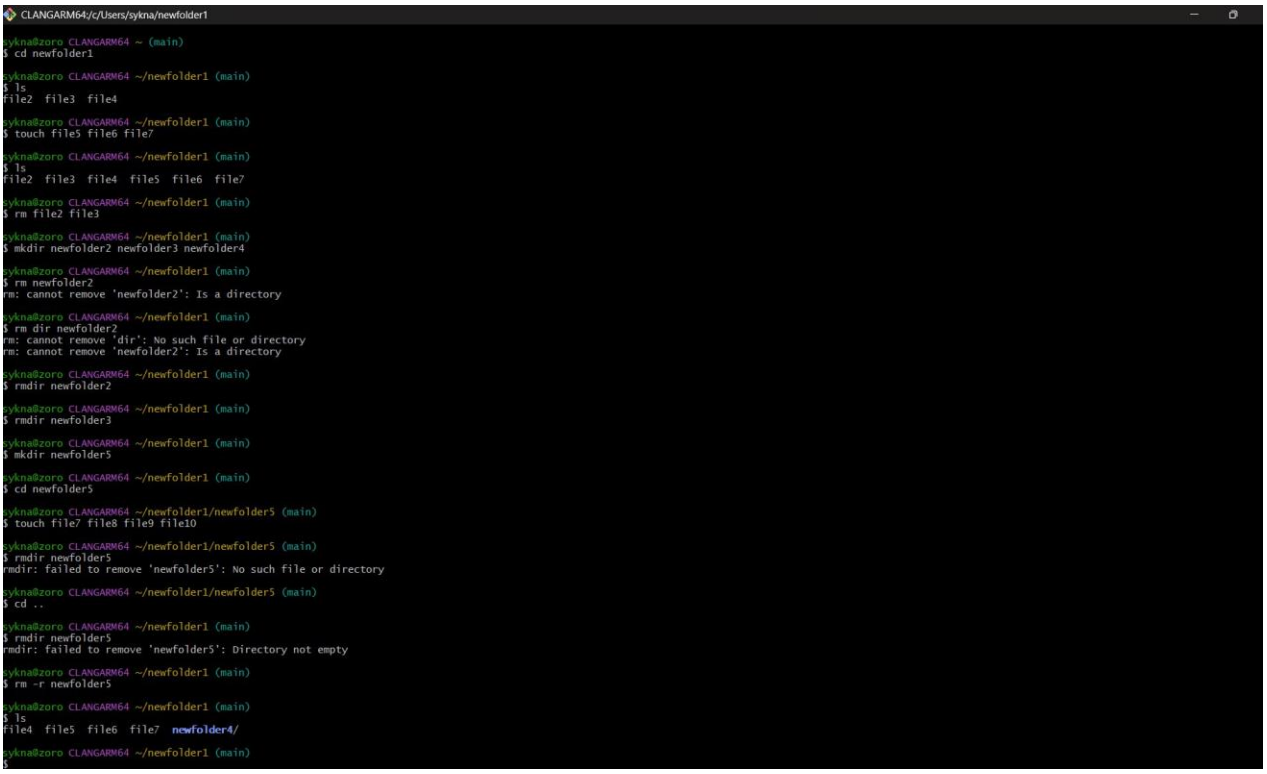
Command:

```
cd ..
```

Description:

Moves up one level to the parent directory. Used in the lab to navigate back out of newfolder5.

Output (Screenshot):



2.3 Listing Files and Folders

Command:

```
ls
```

Description:

Lists all files and folders in the current directory. Used after every operation to confirm that files were created or deleted correctly.

Output (Screenshot):

2.4 Creating Files

Command:

```
touch file1
```

Description:

Creates a single empty file named "file1". If the file already exists, it simply updates its timestamp.

Output (Screenshot):

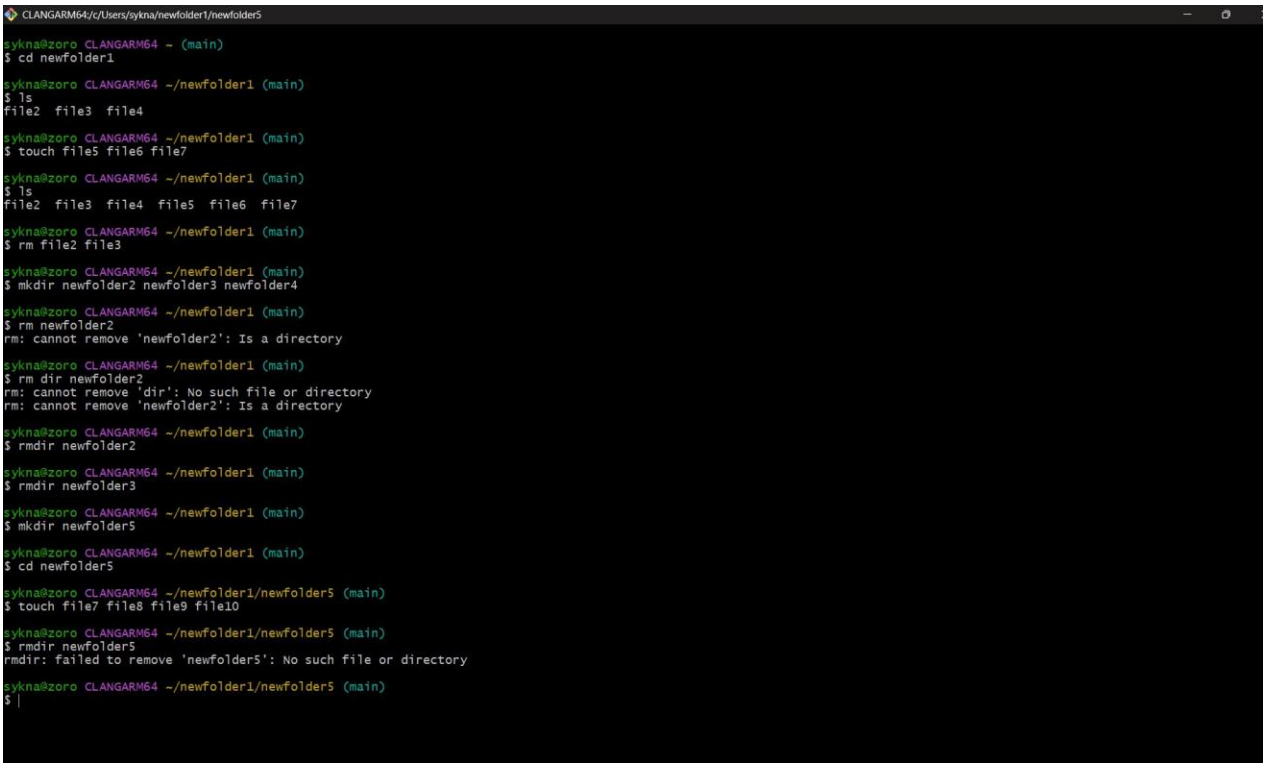
Command:

```
touch file5 file6 file7
```

Description:

Creates multiple empty files at once by listing their names separated by spaces.

Output (Screenshot):



2.5 Deleting Files

Command:

```
rm file1
```

Description:

Permanently deletes the file "file1". There is no Recycle Bin — the file is gone immediately.

Output (Screenshot):

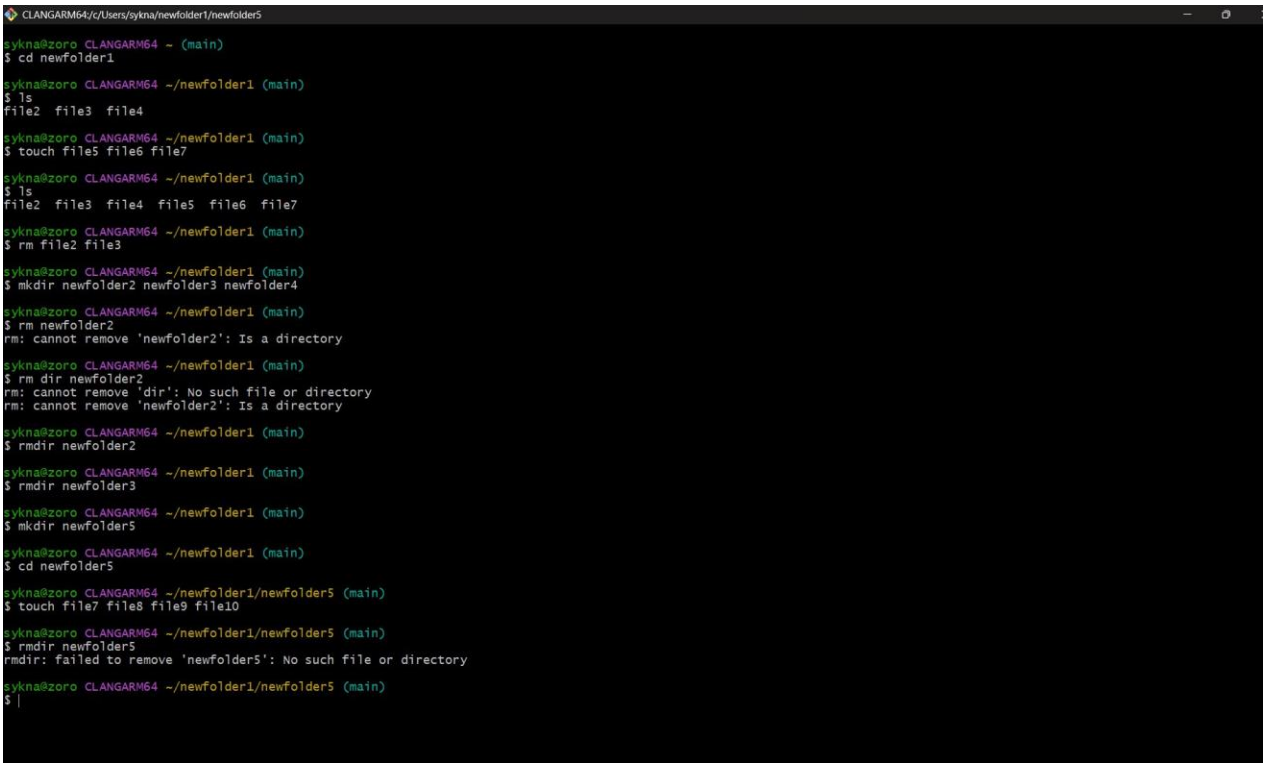
Command:

```
rm file2 file3
```

Description:

Deletes multiple files at once by listing them separated by spaces.

Output (Screenshot):



2.6 Creating Folders

Command:

```
mkdir newfolder1
```

Description:

Creates a new directory named "newfolder1" in the current location.

Output (Screenshot):

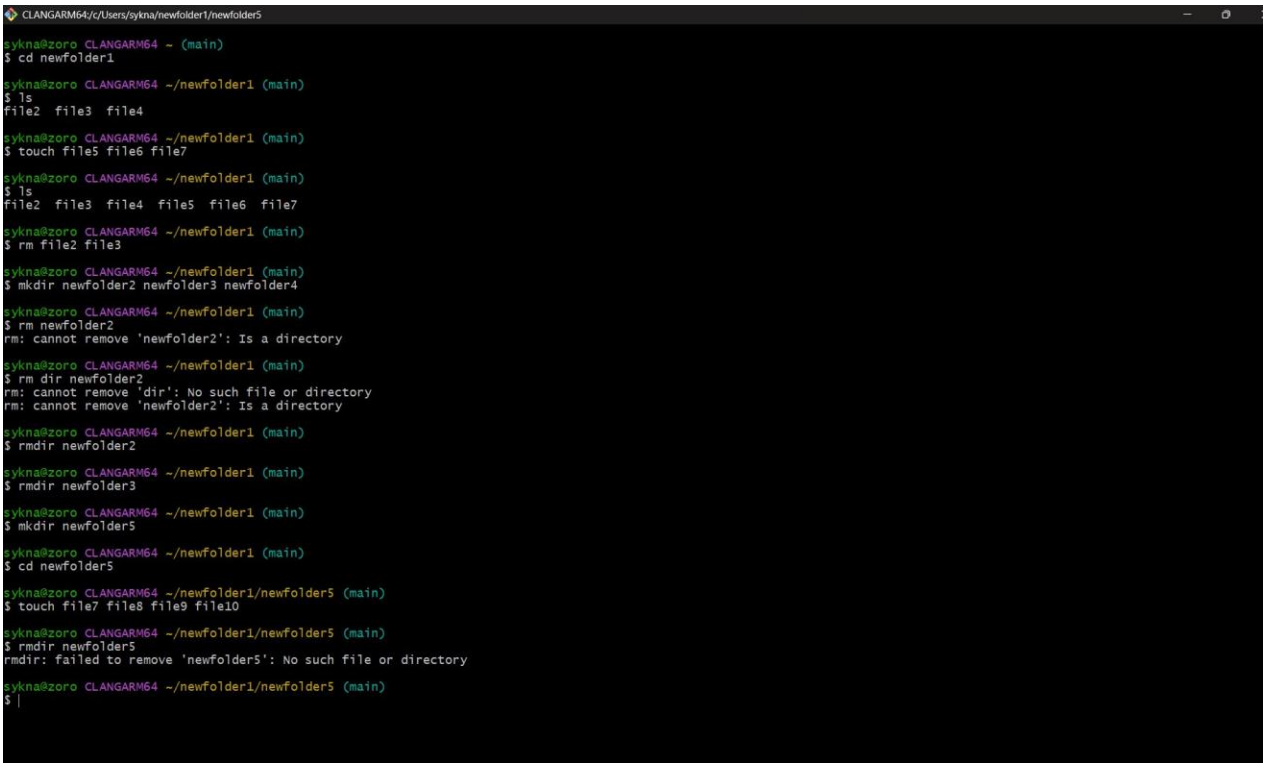
Command:

```
mkdir newfolder2 newfolder3 newfolder4
```

Description:

Creates multiple directories simultaneously in a single command.

Output (Screenshot):



2.7 Deleting Folders

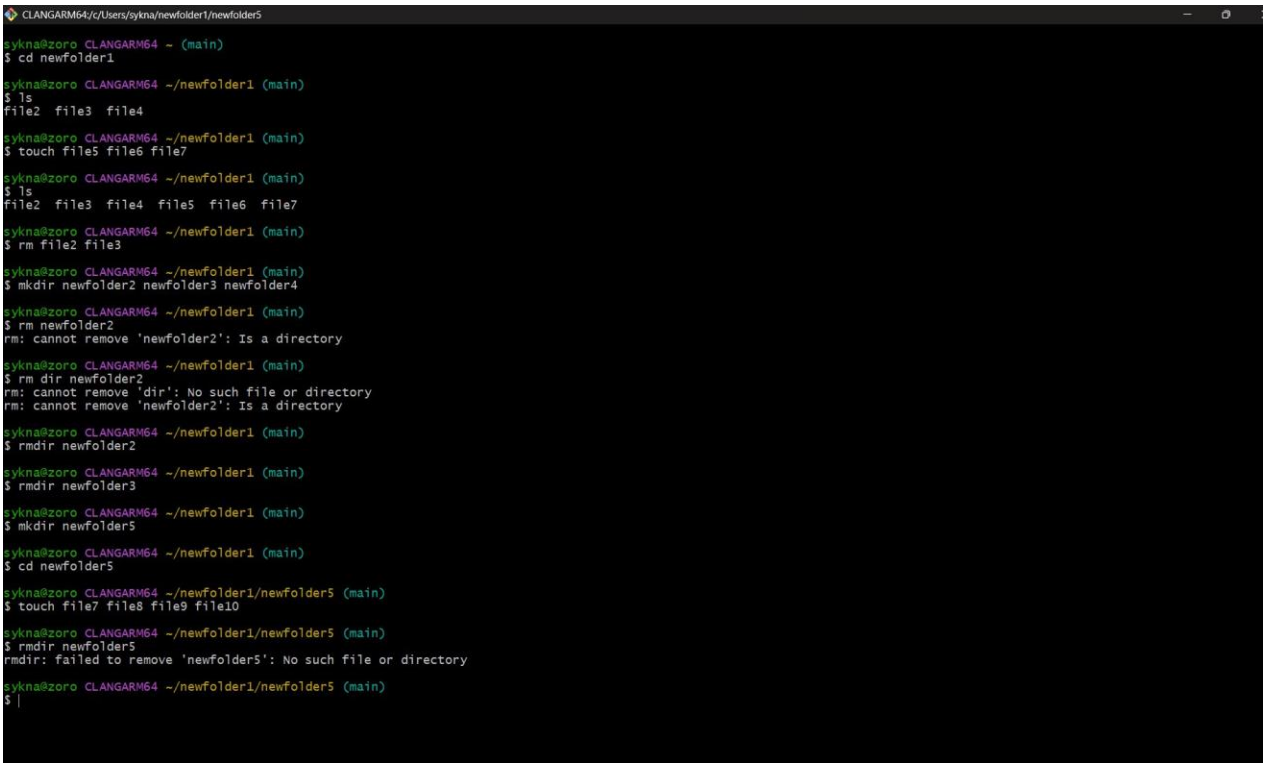
Command:

```
rmdir newfolder2
```

Description:

Removes an empty directory. If the folder contains files, rmdir will fail with an error.

Output (Screenshot):



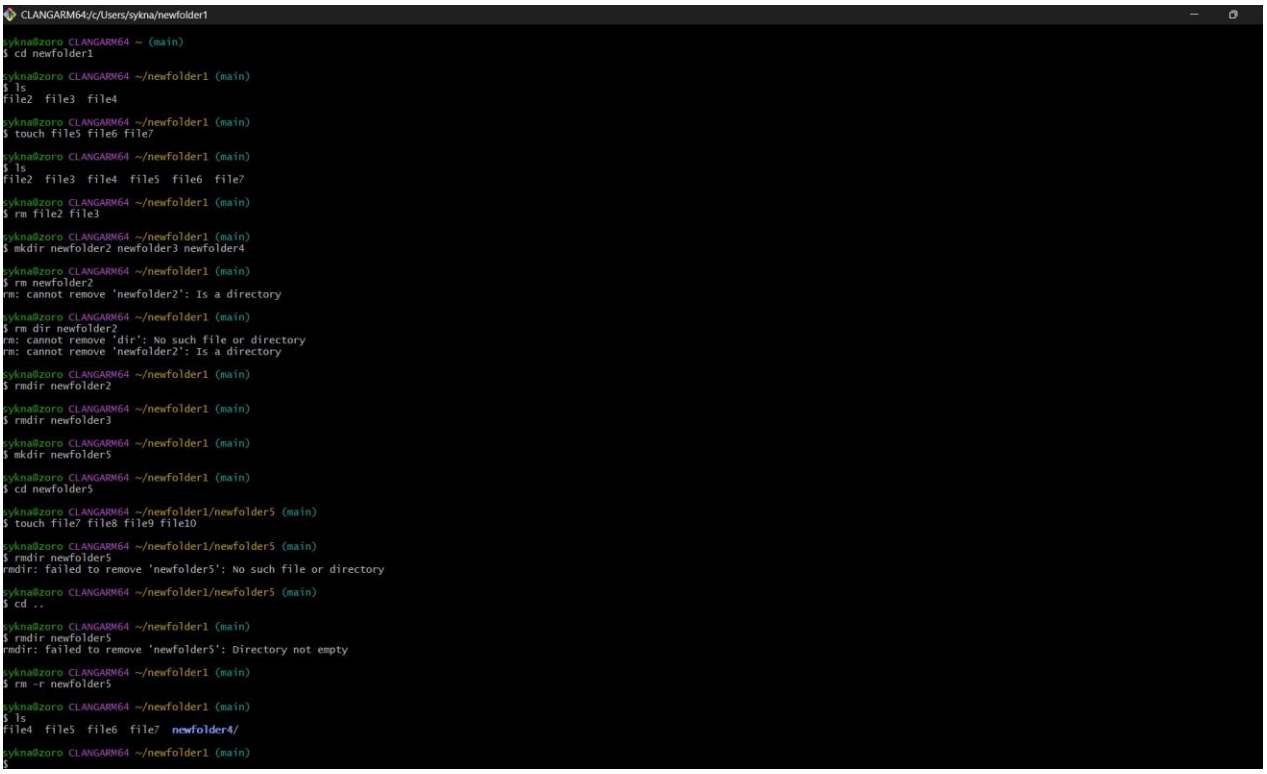
Command:

```
rmdir newfolder5
```

Description:

Attempted to remove newfolder5 with rmdir but it failed because the folder contained files. This showed the limitation of rmdir.

Output (Screenshot):



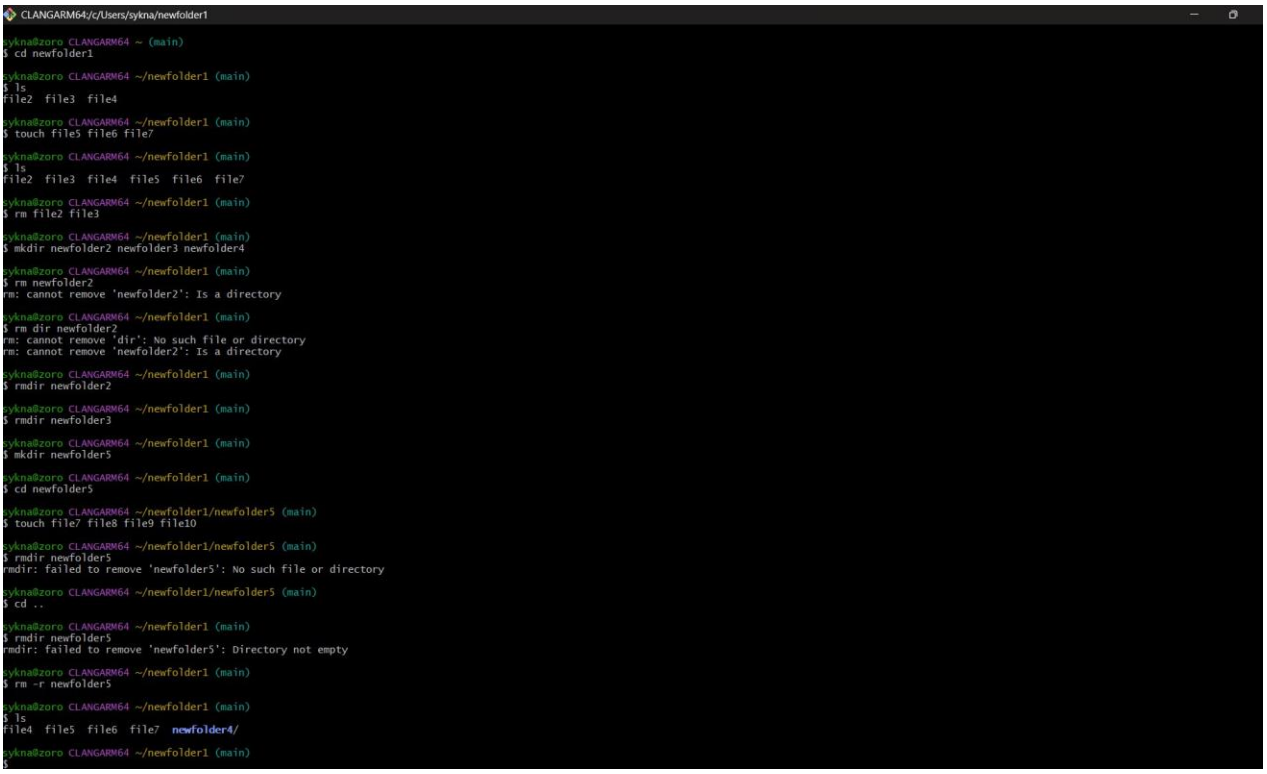
Command:

```
rm -r newfolder5
```

Description:

Forcefully deletes a directory and all its contents recursively. The -r flag stands for recursive. This was the solution after rmdir failed.

Output (Screenshot):



2.8 Writing to Files Using echo

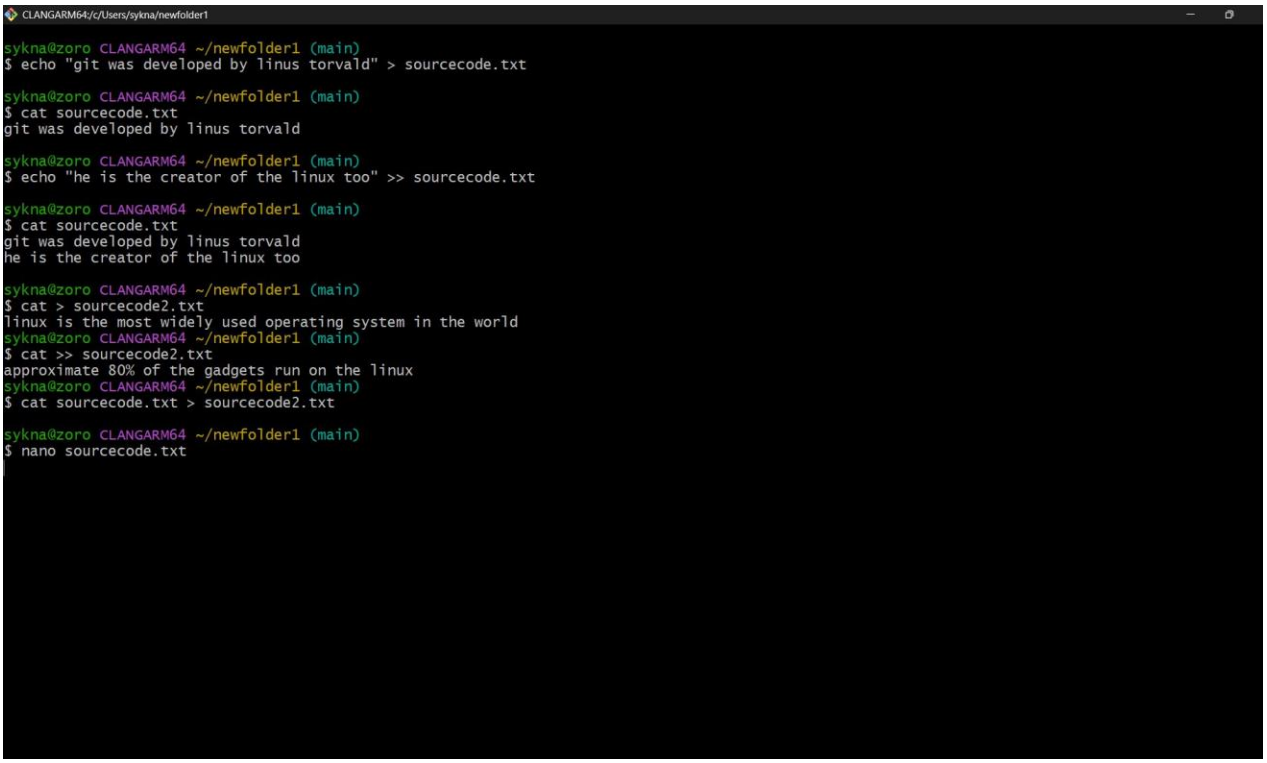
Command:

```
echo "git was developed by linus torvald" > sourcecode.txt
```

Description:

Writes text into sourcecode.txt, creating it if it does not exist. The > operator overwrites any existing content.

Output (Screenshot):



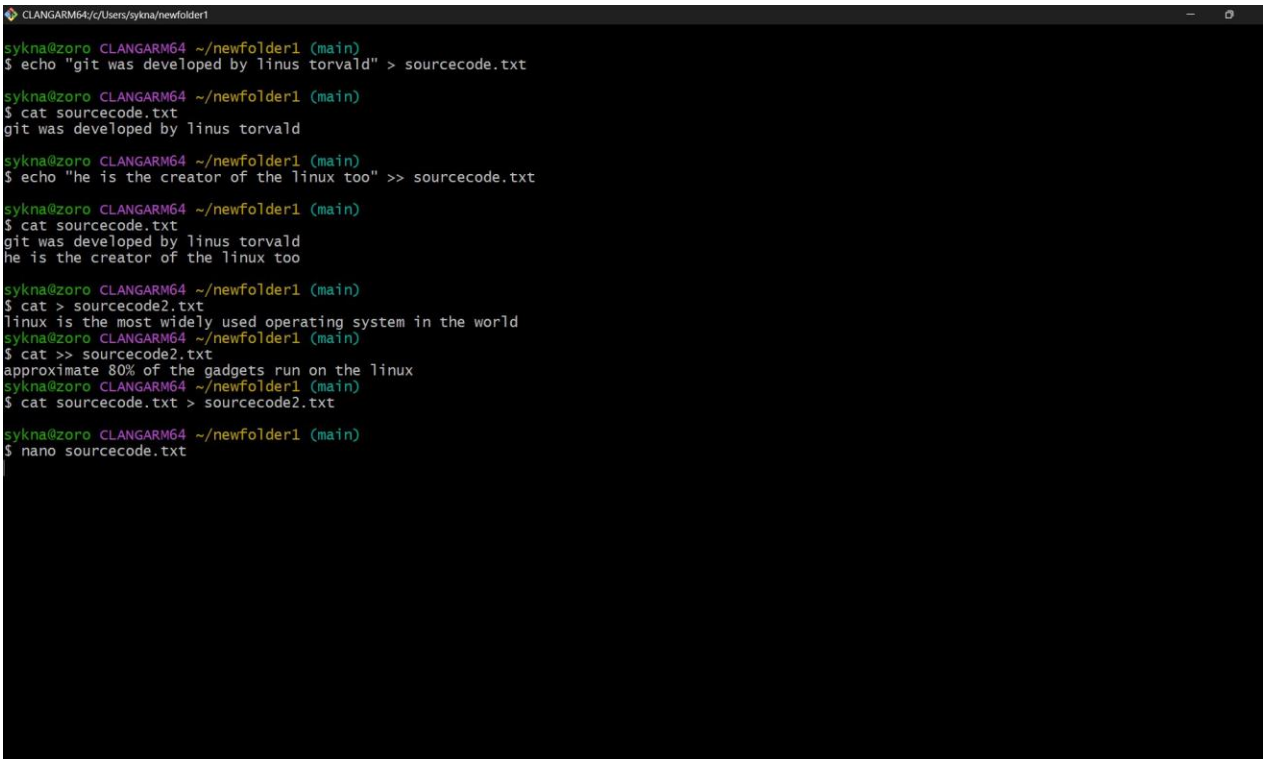
Command:

```
echo "he is the creator of the linux too" >> sourcecode.txt
```

Description:

Appends a new line to sourcecode.txt without deleting existing content. The >> operator adds to the file instead of replacing it.

Output (Screenshot):



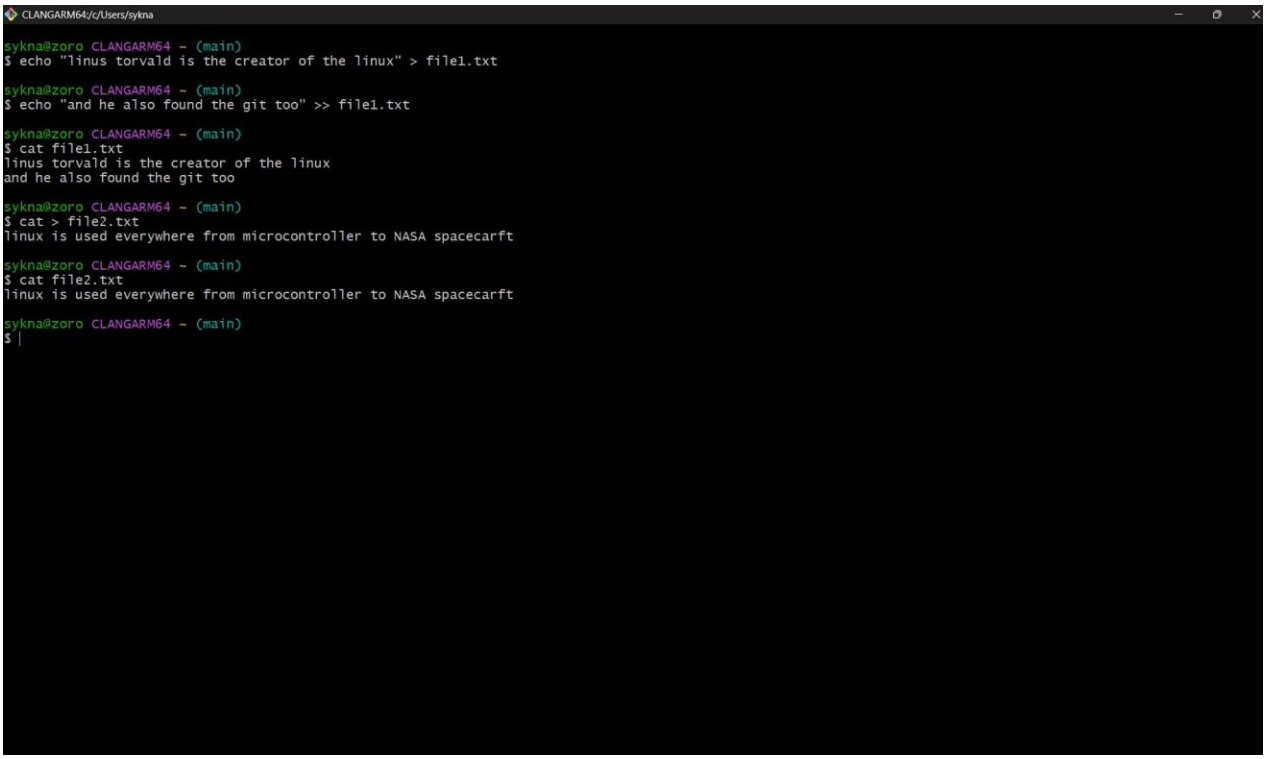
Command:

```
echo "linus torvald is the creator of the linux" > file1.txt
```

Description:

Creates file1.txt and writes the first line into it.

Output (Screenshot):



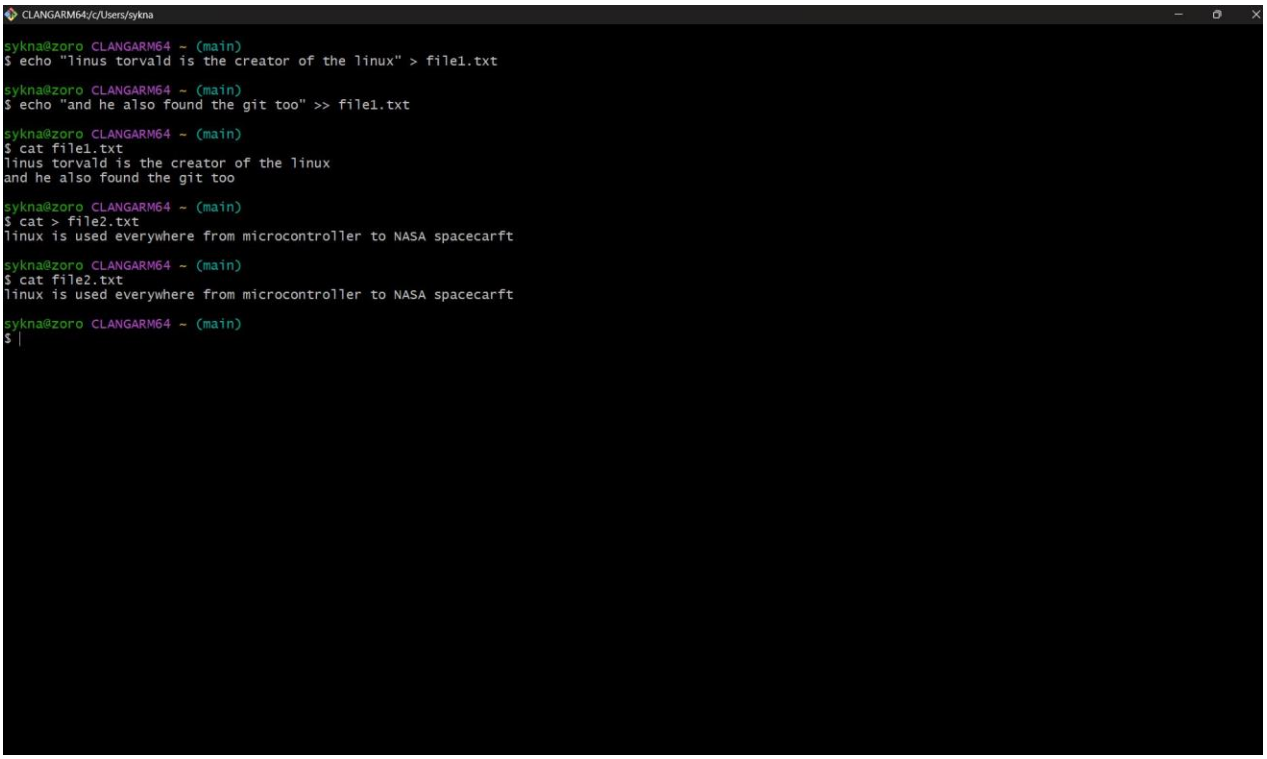
Command:

```
echo "and he also found the git too" >> file1.txt
```

Description:

Appends a second line to file1.txt. Running cat file1.txt afterwards confirmed both lines were present.

Output (Screenshot):



2.9 Reading File Content Using cat

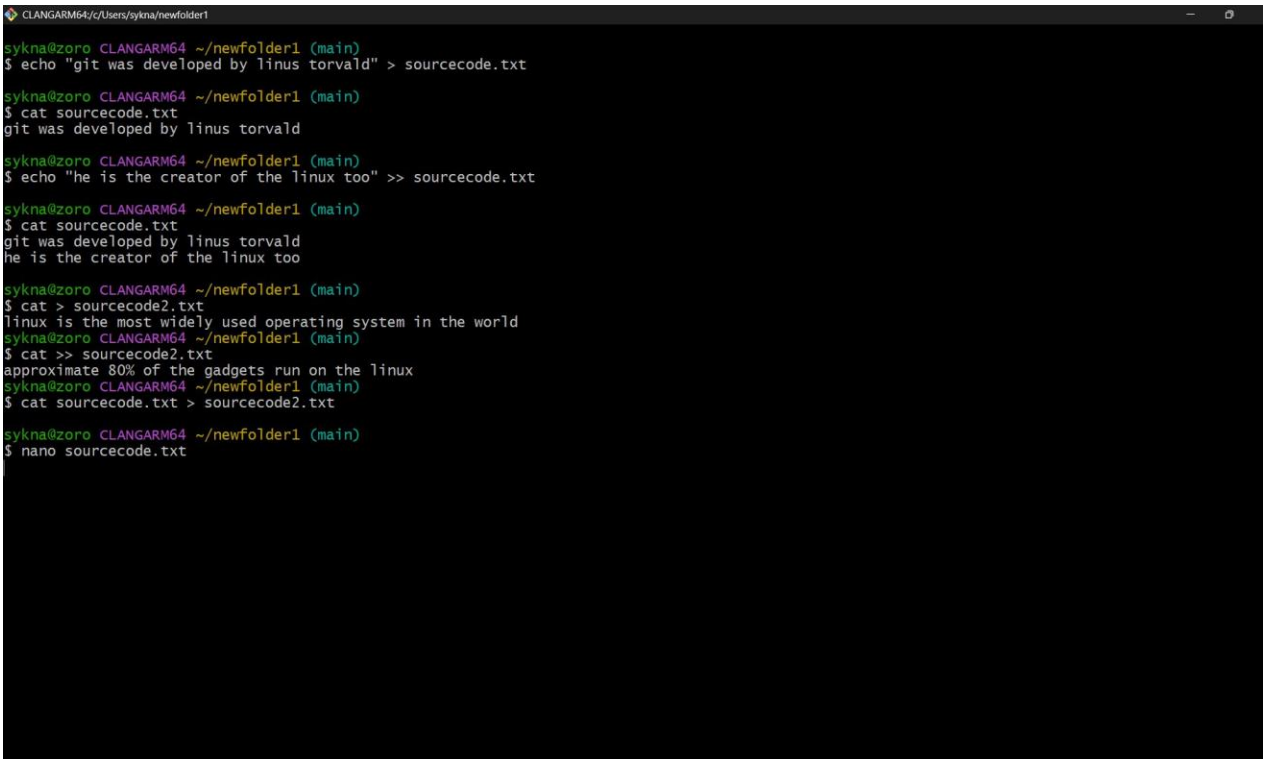
Command:

```
cat sourcecode.txt
```

Description:

Displays the full content of sourcecode.txt in the terminal. Used after every write operation to verify the file content.

Output (Screenshot):



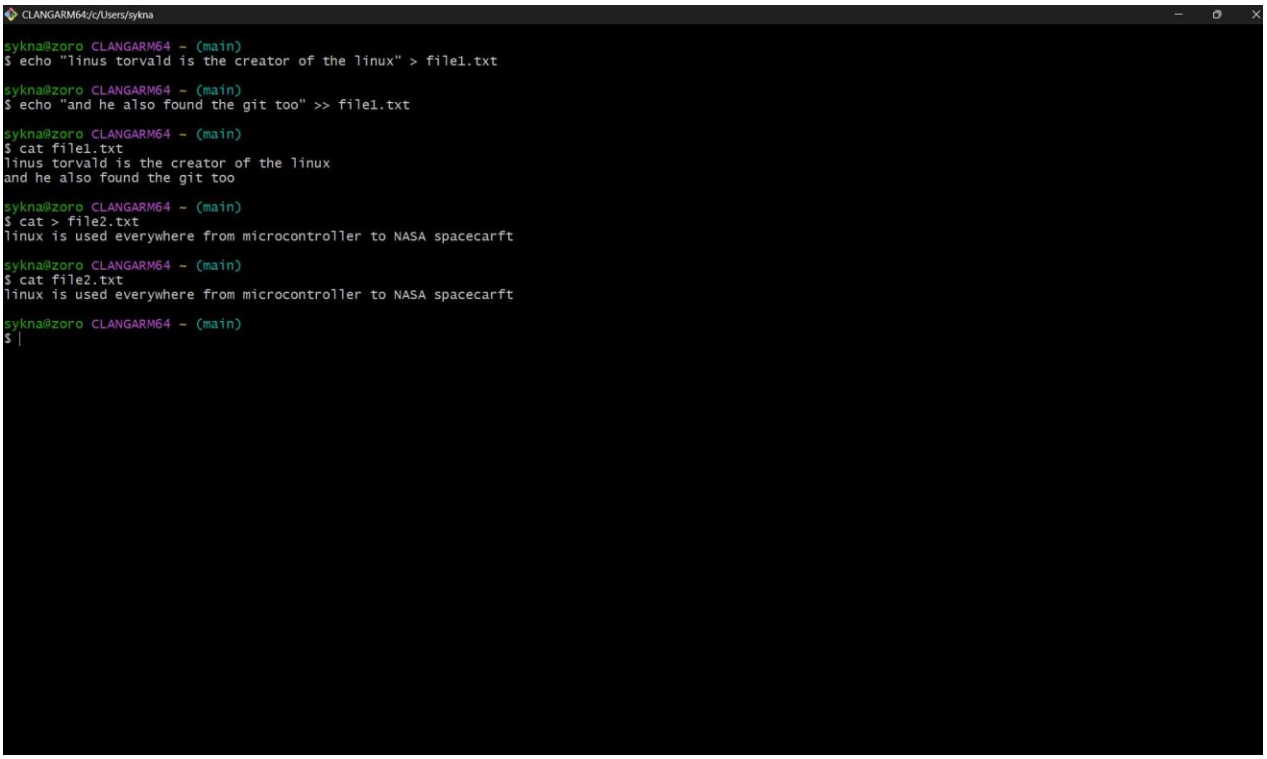
Command:

```
cat file1.txt
```

Description:

Reads and prints the contents of file1.txt to the terminal.

Output (Screenshot):



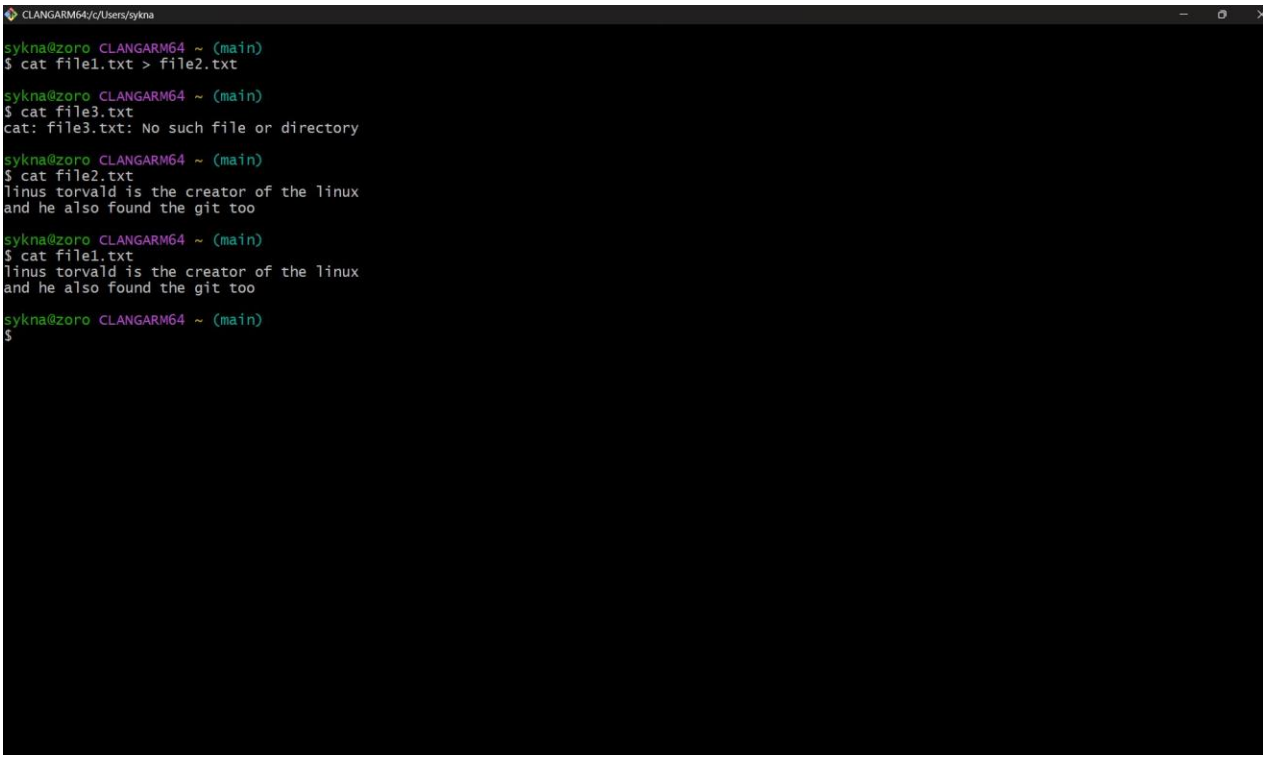
Command:

```
cat file2.txt
```

Description:

Displays the content of file2.txt. Used after writing and copying to verify results.

Output (Screenshot):



2.10 Writing to Files Using cat

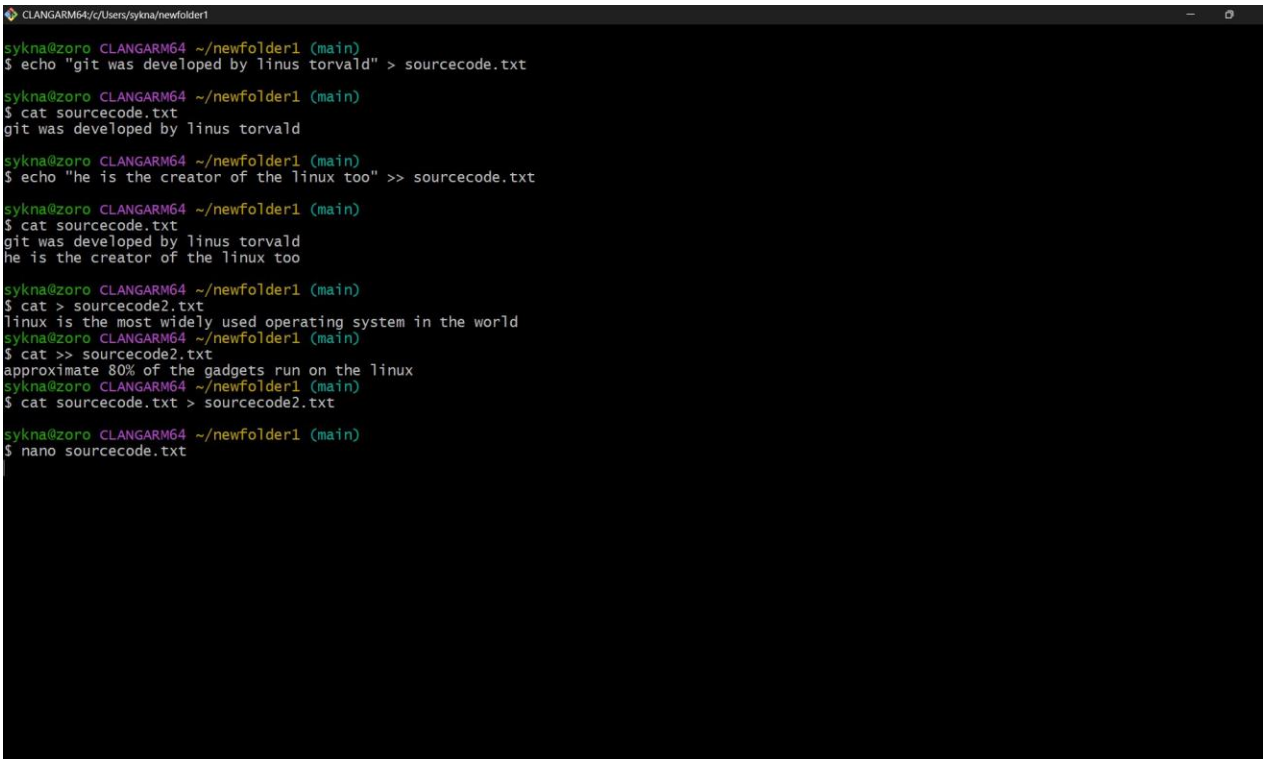
Command:

```
cat > sourcecode2.txt
```

Description:

Creates sourcecode2.txt and opens an input prompt in the terminal. The user types text directly and presses Ctrl+D to save and exit.

Output (Screenshot):



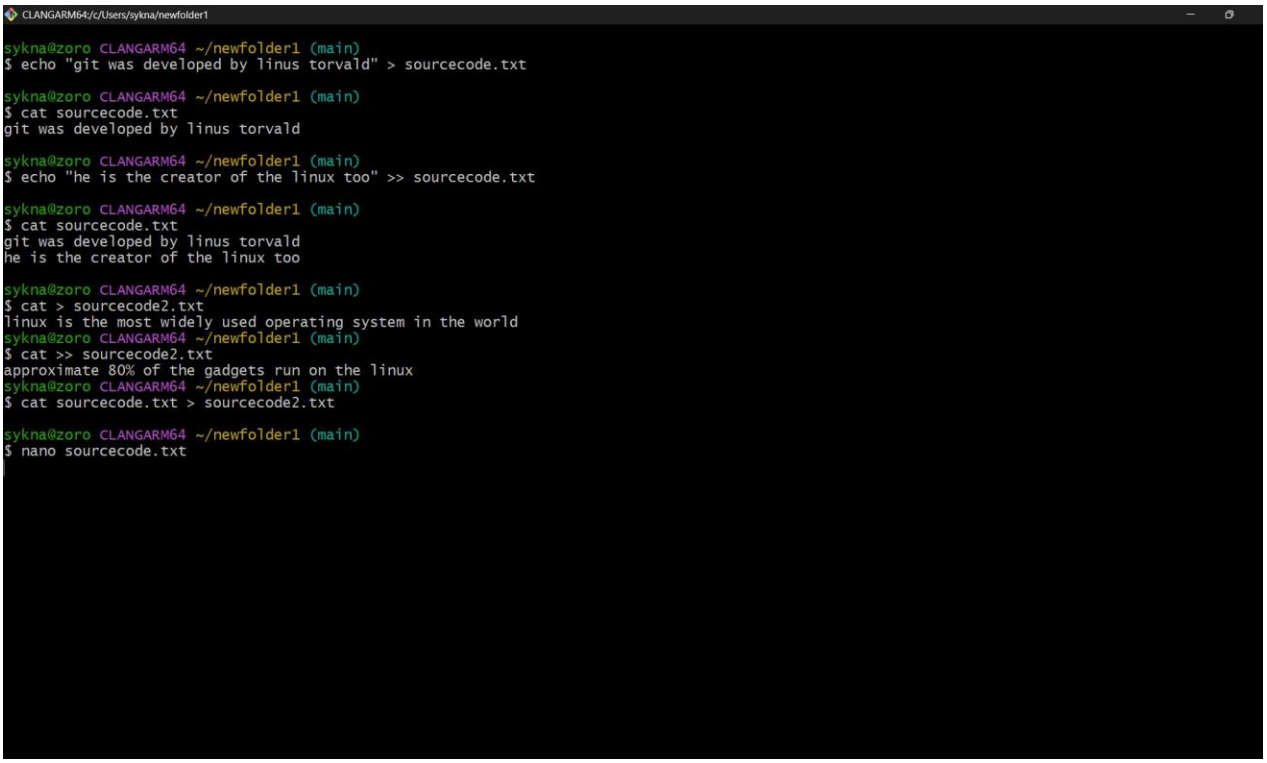
Command:

```
cat >> sourcecode2.txt
```

Description:

Opens sourcecode2.txt in append mode. Any text typed will be added to the end of the file. Ctrl+D saves and exits.

Output (Screenshot):



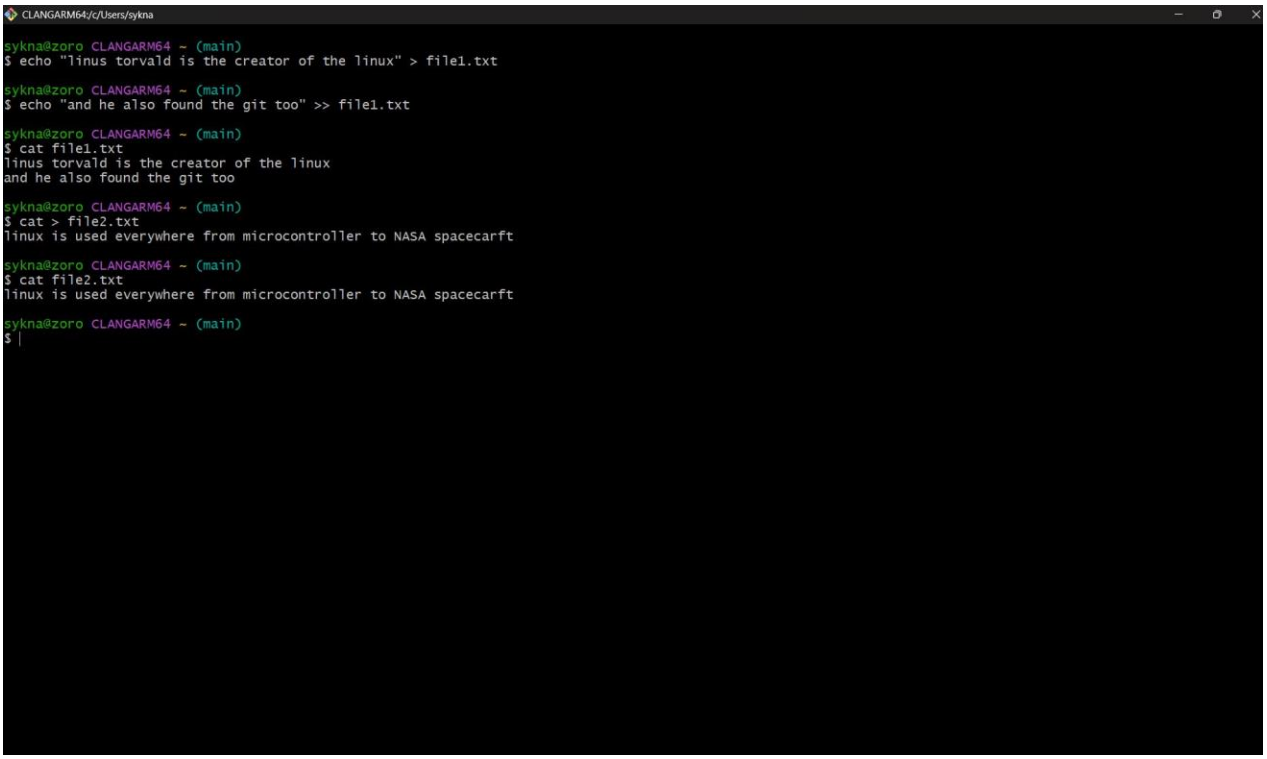
Command:

```
cat > file2.txt
```

Description:

Creates file2.txt interactively. The user typed a line about Linux and saved it using Ctrl+D.

Output (Screenshot):



2.11 Copying File Content

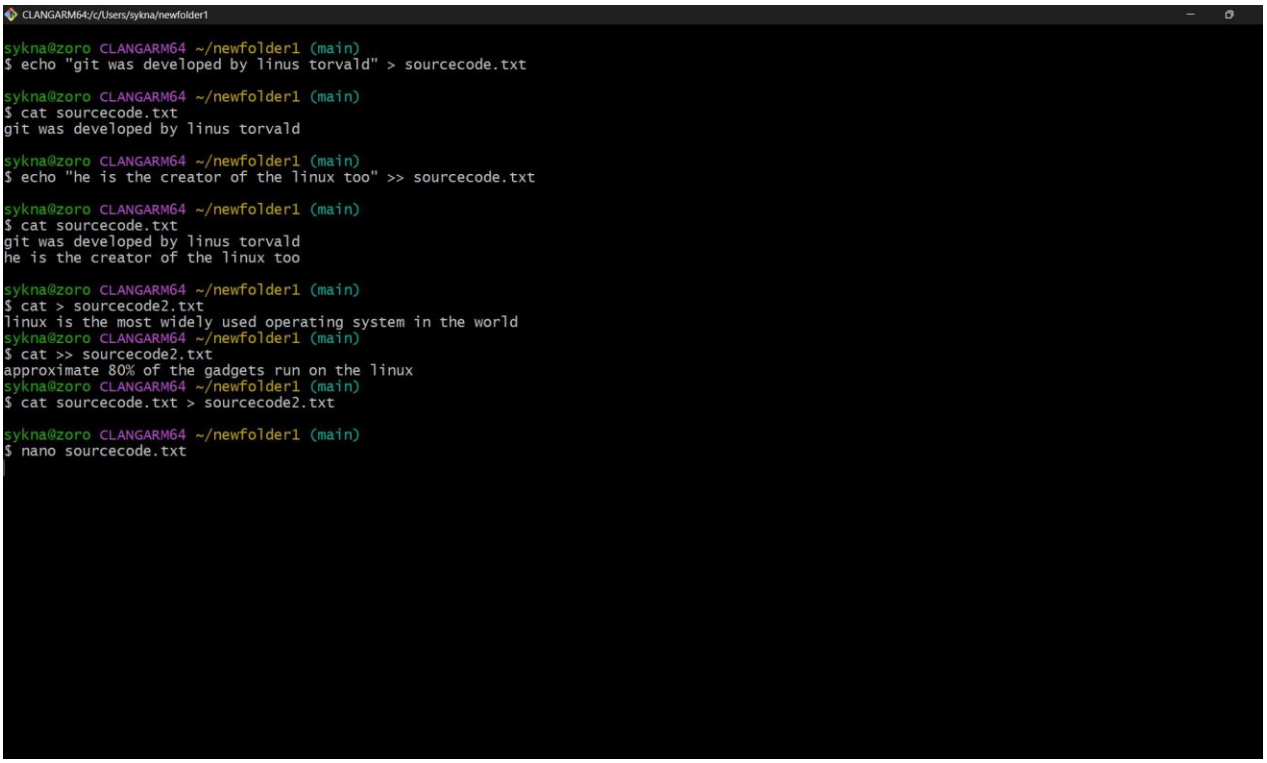
Command:

```
cat sourcecode.txt > sourcecode2.txt
```

Description:

Copies the entire content of sourcecode.txt into sourcecode2.txt, overwriting anything previously in it.

Output (Screenshot):



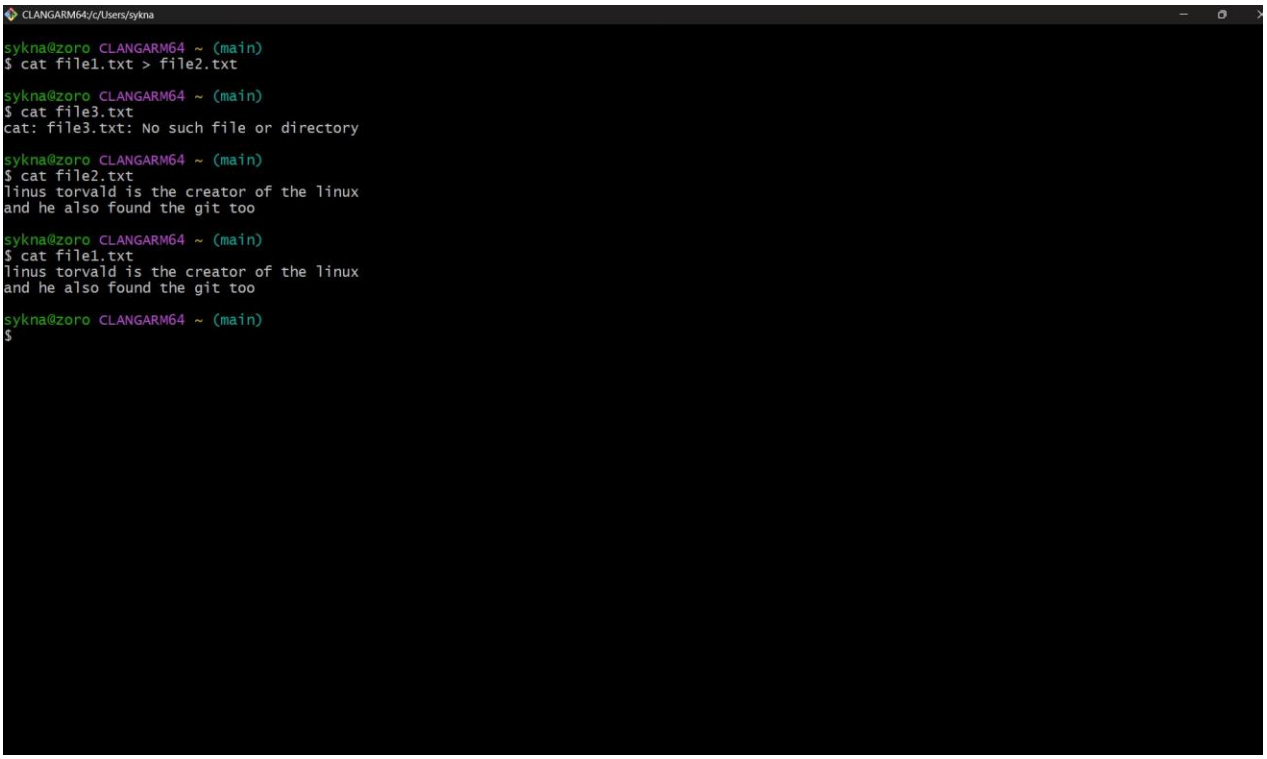
Command:

```
cat file1.txt > file2.txt
```

Description:

Copies all content of file1.txt into file2.txt. Verified afterwards with cat file2.txt.

Output (Screenshot):



2.12 Nano Text Editor

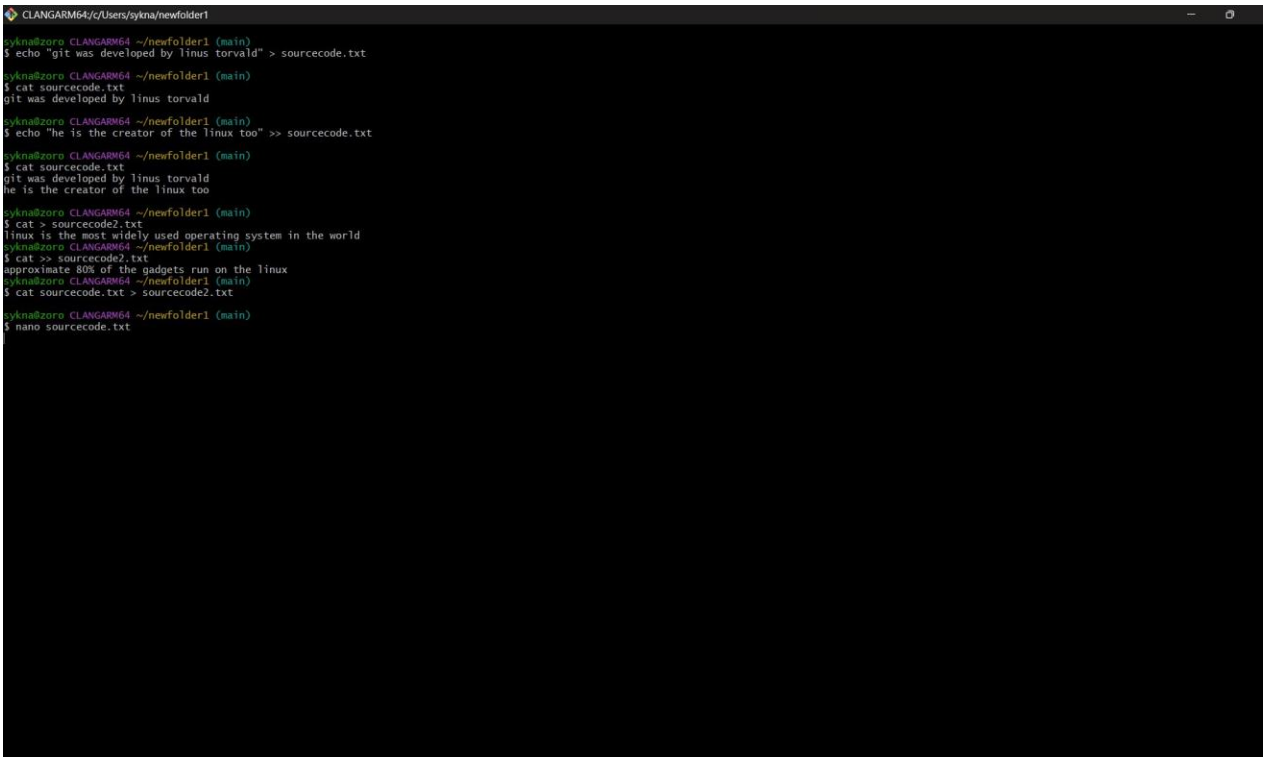
Command:

```
nano sourcecode.txt
```

Description:

Opens sourcecode.txt in the Nano text editor. Nano displays shortcuts at the bottom of the screen. Use Ctrl+O to save and Ctrl+X to exit.

Output (Screenshot):



2.13 Vi Editor Commands

Vi is a command-line text editor with two main modes: Command Mode (default, for navigation and editing) and Insert Mode (for typing text). Press i to enter Insert Mode and Esc to return to Command Mode.

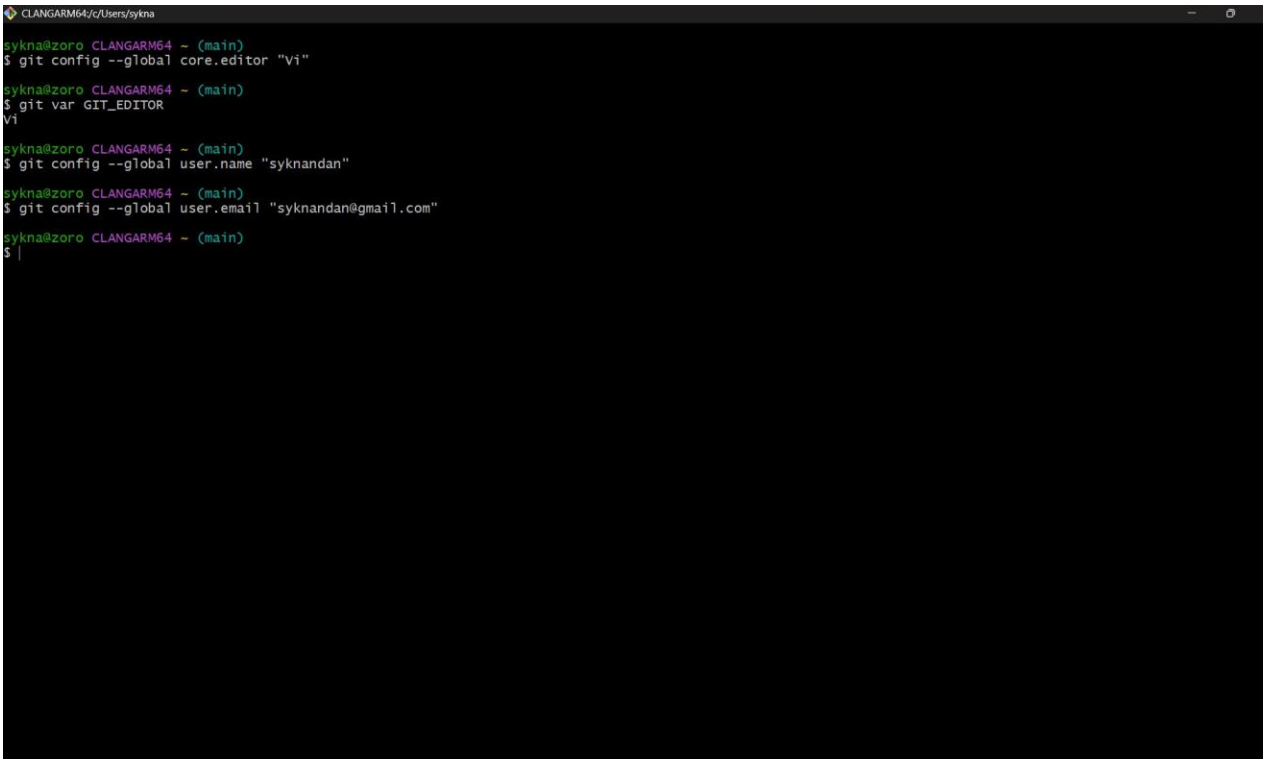
Command:

```
vi filename.txt
```

Description:

Opens a file in vi. If it does not exist, vi creates it. On opening, vi is in Command Mode.

Output (Screenshot):



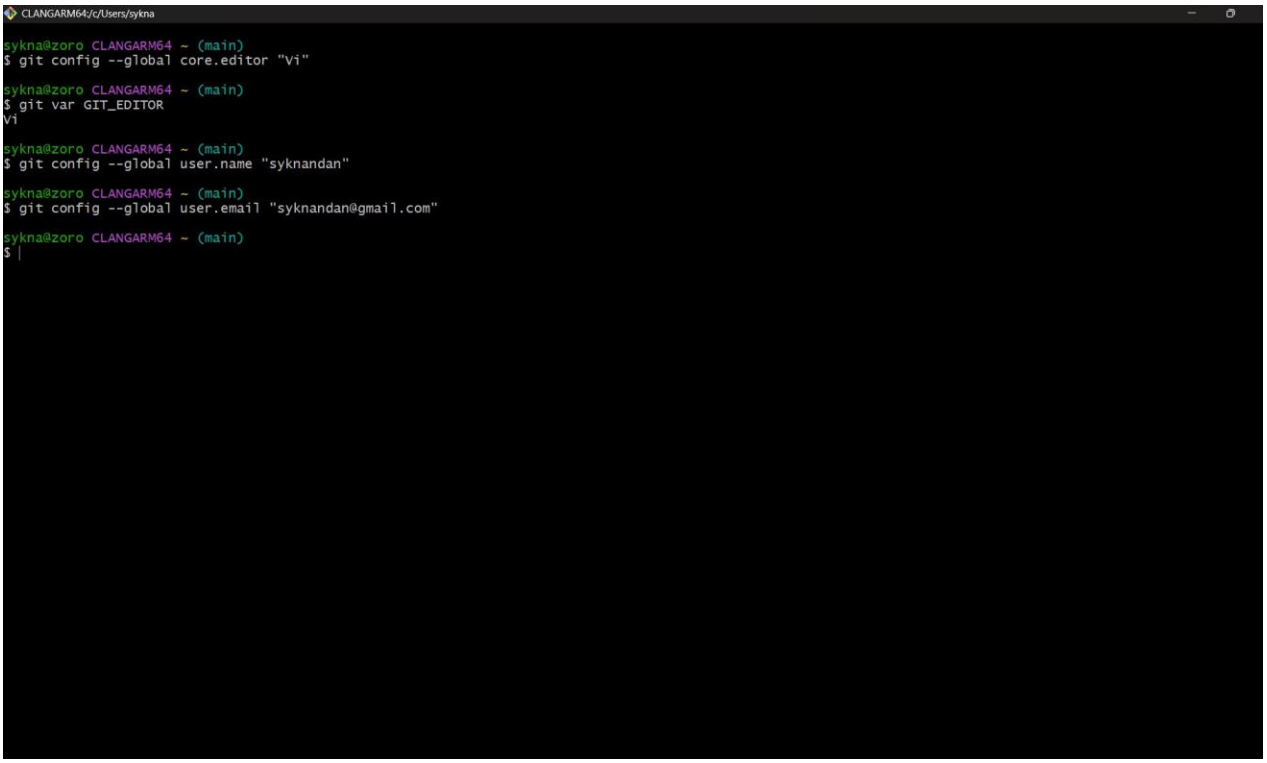
Command:

```
i  →  Enter Insert Mode
```

Description:

Switches from Command Mode to Insert Mode, allowing text input at the cursor position. Other insert keys: a (after cursor), A (end of line), o (new line below), O (new line above).

Output (Screenshot):



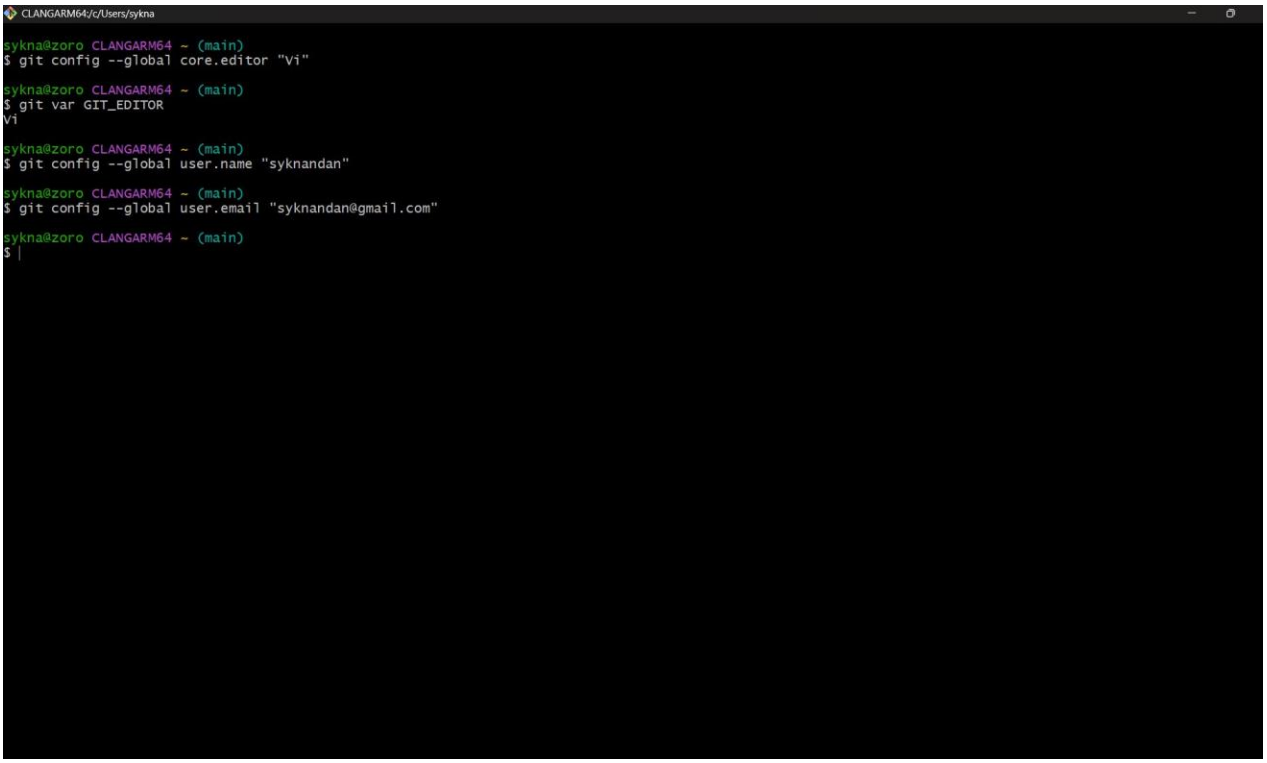
Command:

`:wq` → `Save and Quit`

Description:

After pressing Esc, typing :wq saves the file and exits vi. Other options: :w (save only), :q! (quit without saving), :x (save and quit).

Output (Screenshot):



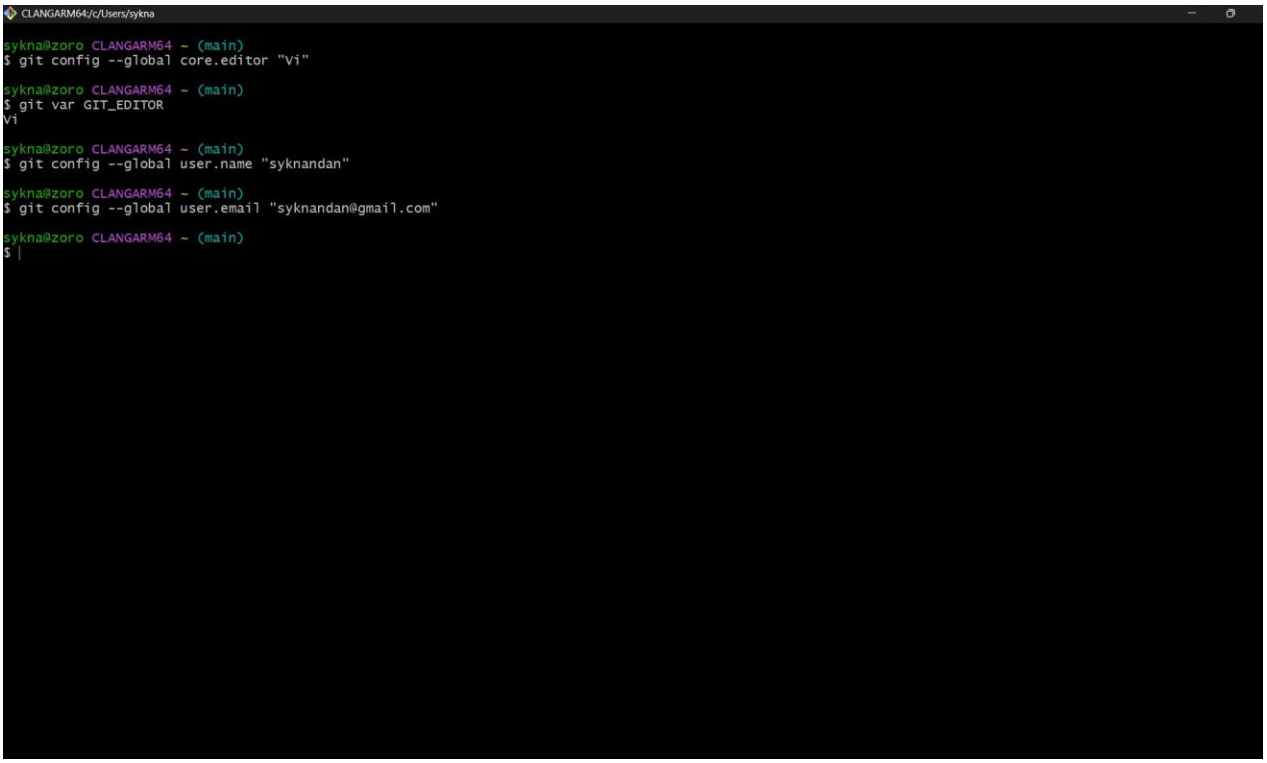
Command:

```
dd  →  Delete Line
```

Description:

In Command Mode, dd deletes the entire current line. Other delete commands: x (delete character), d$ (delete to end of line).

Output (Screenshot):



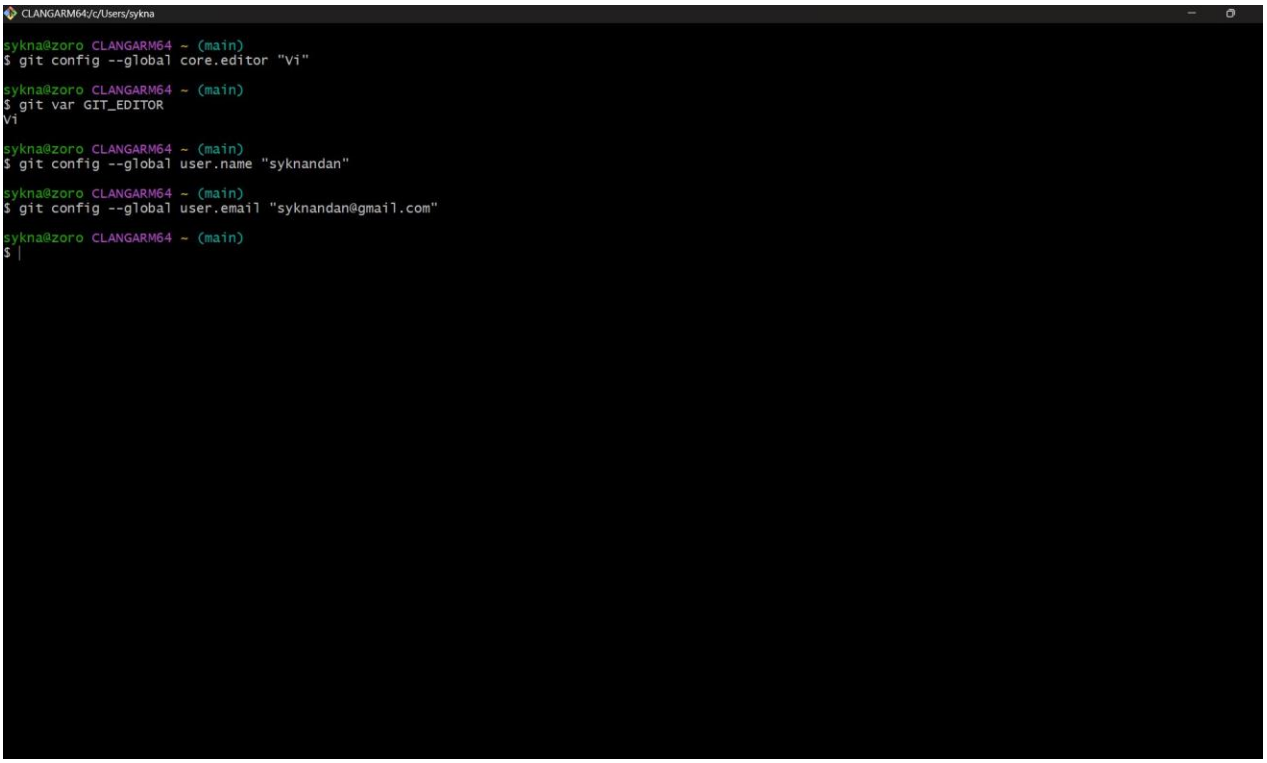
Command:

`yy / p / P  →  Copy and Paste`

Description:

yy copies (yanks) the current line. p pastes it below the cursor. P pastes it above. Used in the lab copy-paste exercise.

Output (Screenshot):



2.14 Git Repository and Branch Commands

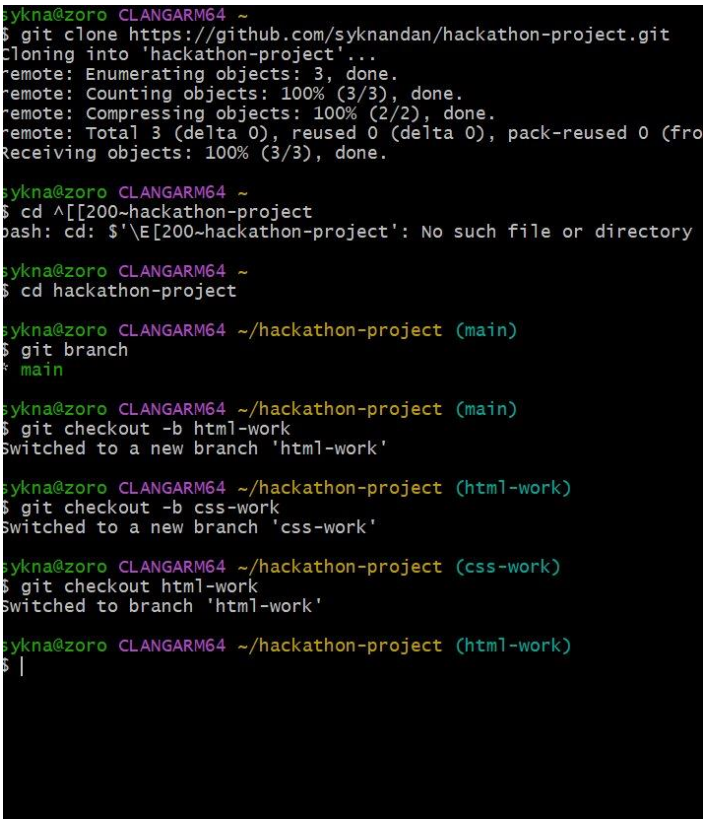
Command:

```
git clone https://github.com/syknandan/hackathon-project.git
```

Description:

Downloads the entire remote repository from GitHub to the local machine, including all files and history.

Output (Screenshot):



Command:

```
git branch
```

Description:

Lists all local branches. The currently active branch is marked with an asterisk (*). Confirmed that "main" was the active branch after cloning.

Output (Screenshot):

```
ykna@zoro CLANGARM64 ~
$ git clone https://github.com/syknandan/hackathon-project.git
Cloning into 'hackathon-project'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (fro
Receiving objects: 100% (3/3), done.

ykna@zoro CLANGARM64 ~
$ cd ^[[200~hackathon-project
bash: cd: $'\E[200~hackathon-project': No such file or directory

ykna@zoro CLANGARM64 ~
$ cd hackathon-project

ykna@zoro CLANGARM64 ~/hackathon-project (main)
$ git branch
* main

ykna@zoro CLANGARM64 ~/hackathon-project (main)
$ git checkout -b html-work
Switched to a new branch 'html-work'

ykna@zoro CLANGARM64 ~/hackathon-project (html-work)
$ git checkout -b css-work
Switched to a new branch 'css-work'

ykna@zoro CLANGARM64 ~/hackathon-project (css-work)
$ git checkout html-work
Switched to branch 'html-work'

ykna@zoro CLANGARM64 ~/hackathon-project (html-work)
$ |
```

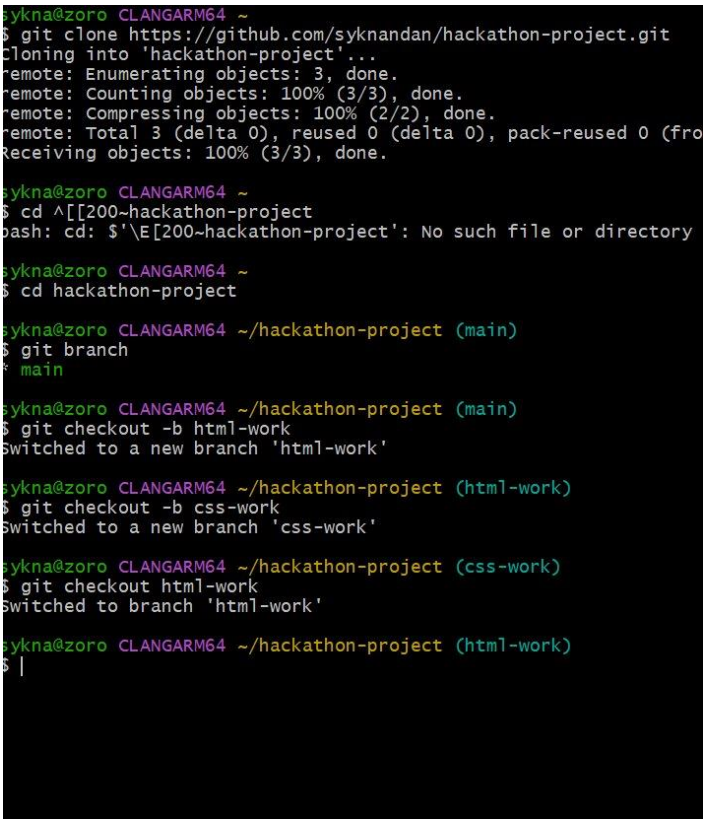
Command:

```
git checkout -b html-work
```

Description:

Creates a new branch named "html-work" and immediately switches to it. The -b flag combines branch creation and switching into one step.

Output (Screenshot):



Command:

```
git checkout -b css-work
```

Description:

Creates another new branch "css-work". Demonstrates creating multiple feature branches for parallel development.

Output (Screenshot):

```
ykna@zoro CLANGARM64 ~
$ git clone https://github.com/syknandan/hackathon-project.git
Cloning into 'hackathon-project'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (fro
Receiving objects: 100% (3/3), done.

ykna@zoro CLANGARM64 ~
$ cd ^[[200~hackathon-project
bash: cd: $'\E[200~hackathon-project': No such file or directory

ykna@zoro CLANGARM64 ~
$ cd hackathon-project

ykna@zoro CLANGARM64 ~/hackathon-project (main)
$ git branch
* main

ykna@zoro CLANGARM64 ~/hackathon-project (main)
$ git checkout -b html-work
Switched to a new branch 'html-work'

ykna@zoro CLANGARM64 ~/hackathon-project (html-work)
$ git checkout -b css-work
Switched to a new branch 'css-work'

ykna@zoro CLANGARM64 ~/hackathon-project (css-work)
$ git checkout html-work
Switched to branch 'html-work'

ykna@zoro CLANGARM64 ~/hackathon-project (html-work)
$ |
```

Command:

```
git checkout html-work
```

Description:

Switches the working directory to the existing "html-work" branch. Used without -b since the branch already exists.

Output (Screenshot):

```
ykna@zoro CLANGARM64 ~
$ git clone https://github.com/syknandan/hackathon-project.git
Cloning into 'hackathon-project'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (fro
Receiving objects: 100% (3/3), done.

ykna@zoro CLANGARM64 ~
$ cd ^[[200~hackathon-project
bash: cd: $'\E[200~hackathon-project': No such file or directory

ykna@zoro CLANGARM64 ~
$ cd hackathon-project

ykna@zoro CLANGARM64 ~/hackathon-project (main)
$ git branch
* main

ykna@zoro CLANGARM64 ~/hackathon-project (main)
$ git checkout -b html-work
Switched to a new branch 'html-work'

ykna@zoro CLANGARM64 ~/hackathon-project (html-work)
$ git checkout -b css-work
Switched to a new branch 'css-work'

ykna@zoro CLANGARM64 ~/hackathon-project (css-work)
$ git checkout html-work
Switched to branch 'html-work'

ykna@zoro CLANGARM64 ~/hackathon-project (html-work)
$ |
```